December 2009
Final Examination

Introduction to Software Systems

COMP-206

December 18, 2009 at 14:00 – 17:00

Examiner: Joseph Vybihal Assoc Examiner: Joerg Kienzle

Student Name:		**McGill ID:**									

INSTRUCTIONS:

- This is a **CLOSED BOOK** examination.
- You are permitted **TRANSLATION** dictionaries ONLY.
- **STANDARD CALCULATOR** permitted ONLY.
- This examination is **PRINTED ON BOTH SIDES** of the paper
- This examination paper **MUST BE RETURNED**
- You are permitted to write your answers in either **English or French**
- Write your answers in the **exam booklet** providedd
- Attemp all questions, **part marks** will be assigned, show your work.

Grading

Section	Grade	Your Mark
Question 1: GNU	25	
Question 2: HTML and CGI	25	
Question 3: C	25	
Question 4: Python	25	
Total	100 %	

Question 1: GNU (25 points)

Answer the following questions about the GNU tool set:

(A) Makefiles

Create a makefile for the following C program. The program consists of multiple source files. It is an implementation of a book store. It has been designed using module programming techniques, like **extern**. Each source file is a module with a header file. The executable file is called BookStore. The source file names are: main.c + main.h, Book.c + Book.h, and Printout.c + Printout.h.

(B) RCS

Given the source files in the book store question from above, and assuming the program is being developed within the folder /home/source, perform the following RCS activities:

First: Create the RCS repository
Second: Put all the source files into the repository
Third: Give permission to mJackson, hPotter, and msOprah to edit all those files
Fourth: Assume two users edited the same file at the same time, how can RCS help us?
Fifth: Print out the log for main.c

How it will be graded:
10 points – The makefile of (A), proportionally graded
15 points – The RCS of (B), 3 points for each step, proportionally graded

Question 2: HTML and CGI (25 points)

Create a login/register screen for a web site. The page will have the following characteristics: blue background, a title in Times New Roman font at 23 point size saying WELCOME in all caps, a table with two rows and two columns, the left column first row will display the title LOGIN in a header size of 2, the right column top row will display the title REGISTER in a header size of 2 with red writing. The second row left column will use a CGI form that calls a program called login.exe launched from the server's default directory. The form will display the prompts USER NAME and PASSWORD with a text box for each. The password text box should not show what the user types. There should be a single submit button called LOGIN. All these controls are in that one cell. The second row right column will contain a CGI form that calls a program called register.py. This is launched from the server's default directory. This cell contains the following user prompts: REAL NAME, USER NAME, PASSWORD and EMAIL. Each of them will have a text box. The password text box should not show what the user typed. There also needs to be two radio buttons that are labeled MALE and FEMALE. Also a submit button labeled REGISTER. At the bottom of the page, centered and in Ariel font, displayed using CSS, a link to help.htm and saying Click Here For Help.

How it will be graded:

4 points – HTML layout	5 points – Fonts	5 points – Table
4 points – CGI layout	5 points – CGI Commands	2 points – CSS

Question 3: C Programming (25 points)

Using the preprocessor and a single C program source file create a program that when compiled and depending on the setting of a #define PROMPT argument will compile the program either as a command-line program or a prompt driven program. When we are compiling the source file in PROMPT mode there must be no code specific to the command-line, and the same goes the other way around.

The program operates this way at the command-line:

$ sort original_filename sorted_filename

Where "sort" is the name of the executable program you are creating. "original_filename" is the name of an unsorted text file of integer numbers. One number on each line of the file. "sorted_filename" is the filename of a text file that will contain all the numbers from the original file but in sorted order, from lowest to highest – each number on its own line.

The prompt driven program will ask the user for the name of the original_filename and the sorted_filename. It will then use the **same** sorting algorithm as the command-line version used.

If the original_filename does not exist the program terminates prematurely with an error message indicating that the file was not found. If the sorted_filename cannot be created or if it already exists then the program terminates prematurely with a message indicating for what reason it stopped early: unable to open file or file already exists.

Create this program.

How it will be graded:

5 points – Prompt code part	5 points – Command-line code part
5 points – **Same** sorting algorithm	5 points – The preprocessor
5 points – The sorting algorithm	

Question 4: PYTHON Programming

Write a Python program that connects with the HTML script from Question 2, above. Your Python program will by called Register.py and is responsible for registering and validating new users. It will assume a comma separated text file database called user.csv residing in the same folder as the program. This file will contain one user per row with their information separated by the comma delineator. The entire record will be terminated by the CR/LF combination, as is standard in CSV files. The data stored in this database will be in the following order: user name, password, real name, email, user type. The "user type" field can contain only two values: regular (for normal users) or sysop (for administrators).

Your Python program must do the following:

1. Receive data from the CGI script
2. Verify if user.csv exists, if not, it will create it.
3. Verify if the user name already exists
   - If it does exist, then the program displays an error message on the user's browser.
   - Program terminates
4. If it gets here then everything is okay so the new record is added to the CSV. The user type "regular" is assumed and automatically added.
5. The program then displays on the user's browser that they have successfully registered.
6. The program terminates.

How it will be graded:

5 points – CGI interfacing
5 points – User name validation
5 points – Output to browser
5 points – CSV file I/O
5 points – user.csv validation